



Politechnika
Śląska



UCZELNIA
BADAWCZA
INICJATYWA ODSZERKOWAŁA

Katedra Grafiki Wizji Komputerowej
i Systemów Cyfrowych

RAu6

Rok akademicki:

Rodzaj studiów*: SSI/NSI/NSM

Przedmiot (Języki
Asemblerowe/SMiW):

Grupa

Sekcja

2025/2026

SSI

SMiW

4

8

Imię:

Maciej

Nazwisko:

Jazłowiecki

Prowadzący:

OA/JP/KT/GD/GB/KH/AO

JP

Raport końcowy z projektu SMiW

Temat projektu:

Biometryczny system rejestracji czasu pracy/nauki/odpoczynku.

Główne założenia projektu:

System automatycznie identyfikuje użytkownika poprzez odczyt linii papilarnych - rejestruje przy tym czas (rzeczywisty) pobrania odcisku wejściowego/wyjściowego. Pozwala na zapis danych w chmurze i/lub zew. urządzeniu pamięci.

- Tryb praca (rejestracja czasu pracy od początku zeskanowania odcisku wejściowego)
- Tryb nauka (rejestracja czasu potrzebnego na naukę)
- Tryb odpoczynek (po ustalonym czasie, w trybie praca lub nauka - urządzenie powiadamia o czasie na przerwę)

Tryb odpoczynek ma możliwość przedłużenia o odpowiednią ilość czasu - czas ten jest rejestrowany.

Dodatkowo system musi:

- mierzyć czas rzeczywisty za pomocą zewnętrznego RTC
- wyświetlać komunikaty na wyświetlaczu (+ sygnał dioda LED/buzzer)
- zapisywać informacje o trybie/czasie pracy w logu który może zostać wyeksportowany

Zasilanie:

- zasilanie z zasilacza sieciowego
- moduł RTC powinien mieć własny system podtrzymania baterijnego

Data oddania:
dd/mm/rrrr

11/02/2026

Analiza i wybór elementów projektu

Temat: Biometryczny system rejestracji czasu pracy/nauki/odpoczynku.

Celem projektu jest zaprojektowanie i implementacja systemu, który będzie pełnił funkcję rozszerzonego rejestratora czasu pracy (RCP) z autoryzacją biometryczną. System ma automatycznie identyfikować użytkownika za pomocą linii papilarnych i rejestrować czas rozpoczęcia i zakończenia różnych trybów aktywności (Praca, Nauka, Odpoczynek). Rejestrowane dane (logi) mają być zapisywane lokalnie oraz opcjonalnie w chmurze.

W tej analizie, przedstawię przegląd dostępnych rozwiązań, porównam kluczowe parametry niezbędnych komponentów oraz przedstawię swój wybór układów/modułów – by ostatecznie przejść do wyboru mikrokontrolera oraz przedstawić wyjściowy schemat blokowy.

1. Analiza i Wybór Układów Peryferyjnych (Wejście/Wyjście)

Moduł Autoryzacji Biometrycznej: Czytnik Linii Papilarnych

Głównym zadaniem modułu jest autoryzacja i identyfikacja użytkownika poprzez odczyt i weryfikację odcisku palca. W założeniu przyjąłem dokładność w wysokości min. 500 DPI. Zazwyczaj stosuje się dwa główne typy czytników, które różnią się technologią skanowania i sposobem integracji.

	Zalety:	Wady:
Optyczny	Najtańszy, odporny na zarysowania, łatwo dostępny w modułach .	Podatny na zanieczyszczenia zazwyczaj większy fizycznie.
Pojemnościowy	Najwyższa dokładność i bezpieczeństwo, małe rozmiary, szybki odczyt, wysoka rozdzielczość	Droższy, bardziej podatny na uszkodzenia elektrostatyczne/zarysowania.

Tab 1. Porównanie typów czytników

Wstępnie wybrałem kilka modeli cechujących się dokładnością min. 500 DPI oraz są gotowymi modułami – co znacznie ułatwia integrację.

Model	Typ	Interfejs Komunikacji	Cena/szt NETTO	Cechy	Datasheet
Waveshare 22059	Pojemnościowy	UART	96,75 zł	Wbudowany mikrokontroler do identyfikacji z ARM Cortex M3, duża dokładność 508 DPI, mało popularny wybór	Link
R307/R307S	Optyczny	UART	51,39zł	Bardzo popularny wybór w projektach, rozdzielczość 500 DPI, duża baza odcisków - 1k	Link

AS608	Optyczny	UART	44,79zł	Wbudowany algorytm rozpoznawania odcisków palca, rozdzielczość 500 DPI, niska dostępność	Link
DF-SEN0188	Optyczny	UART	151.72zł	mają szerokie wsparcie w postaci gotowych bibliotek np. biblioteka Adafruit Fingerprint, 508 DPI, drogi	Link

Tab 2. Porównanie modeli czytników

Wybrany Element i uzasadnienie:

Moduł Czytnika Linii Papilarnych Optyczny GROW R307. Jego napięcie zasilania to 4.2V do 6V – a typowy pobór prądu wynosi 50mA. Posiada dużą bazę szablonów (w porównaniu do pozostałych modułów – 1000 odcisków), a komunikacja odbywa się poprzez interfejs UART. Jest gotowym modułem, który realizuje wszystkie zaawansowane funkcje biometryczne GenImg, Img2Tz, RegModel, Search. Wymaga 2 linii GPIO. Dodatkowo jest sporo tańszy niż dostępny w małej ilości DF-SEN0188.

Wyświetlacz (Interfejs użytkownika)

Wyświetlacz w moim projekcie ma za zadanie prezentować użytkownikowi menu (Wejście, Wyjście), status operacji a także datę i czas. Potrzebuję niedużego wyświetlacza graficznego z komunikacją – najlepiej z komunikacją I2C.

Model	Technologia/Rozmiar	Rozdzielczość	Interfejs Komunikacji	Cena/szt NETTO	Cechy	Datasheet
TF-051	OLED 1.3"	128x64	I2C v2	45,90 zł	Duży kontrast, głęboka czerń, szybki czas reakcji.	Link
MOD-08246	OLED 1.3"	128x64	I2C v2	29,90zł	Bardzo podobne parametry do MOD-0867, ale niższa cena.	Link
LY190-128064	OLED 0.96"	128 x 64	I2C	28,90zł	Mniejszy fizycznie (0.96"), ale zachowuje rozdzielczość 128x64.	Link

Tab 3. Porównanie modeli wyświetlaczy

Wybrany Element i uzasadnienie:

Wyświetlacz OLED 1.3" 128x64 (MOD-08246). OLED zapewni wyraźny, czytelny, graficzny interfejs, niezbędny do wyświetlania trybów (Praca, Nauka, Odpoczynek), komunikatów oraz bieżącego czasu i daty. Posiada wystarczającą rozdzielczość. Wymaga 2 linii I/O – SDA oraz SDL. Wybór tańszego MOD-08246 jest rozsądny, ponieważ jego parametry są identyczne z droższym TF-051.

Moduły Zegara Czasu Rzeczywistego (RTC)

System wymaga zapewnienia precyzyjnego pomiaru czasu pracy oraz daty. Dodatkowo musi on być podtrzymywany bateryjnie, aby utrzymać czas podczas zaniku głównego zasilania sieciowego.

Model	Interfejs	Zalety	Cena/szt NETTO	Uwagi	Datasheet
DS1307	I2C	Bardzo popularny, prosta biblioteka, niska cena.	4,84zł	Brak kompensacji temperatury, niższa, ale wystarczająca dokładność.	Link
DS3231	I2C	Bardzo wysoka dokładność (TCXO - kompensacja temperaturowa), wbudowany rezonator kwarcowy, wbudowany czujnik temperatury.	24,32zł	Nieco wyższa cena, ale powszechnie dostępny jako moduł.	Link
PCF8523	I2C	Niski pobór prądu, szeroki zakres napięć. (1.0V – 5.0V)	34,07zł	Mniej popularny niż układy Dallas, mniej gotowych bibliotek, drogi	Link
DS1302	SPI (3-wire)	Prosty w obsłudze. Tani.	3,17zł	Wymaga 3 pinów I/O, gorszy od DS3231 pod względem dokładności.	Link

Tab 4. Porównanie modułów RTC

Wybrany Element i uzasadnienie:

Moduł RTC oparty na układzie DS1307 jest łatwo dostępny na rynku i niedrogi. Zapewnia wystarczającą dokładność dla systemu rejestracji czasu aktywności użytkownika. Posiada dużą liczbę gotowych bibliotek oraz komunikuje się z wykorzystaniem I2C – co pozwala oszczędzić liczbę zajętych pinów GPIO.

Pamięć zewnętrzna (EEPROM / Flash)

System ma zapisywać informacje w logu, który może zostać wyeksportowany. Pamięć powinna mieć pojemność minimum 1 MB.

Rozwiązanie	Interfejs	Pojemność (typowa)	Zalety	Wady/Uwagi
Szeregowa pamięć Flash	SPI	0.5MB – 16 MB	Niskie zużycie energii, bardzo szybki zapis odczyt.	Brak możliwości łatwego eksportu logów przez użytkownika.
Karta MicroSD	SPI	Kilka GB	Łatwy eksport logów, duża pojemność	Wymaga użycia dodatkowego gniazda/modułu,
Szeregowa pamięć EEPROM	I2C/SPI	128 kB	Bardzo prosta konstrukcja, 2 piny I2C	Nie spełnia wymogu pojemności

Tab 5. Porównanie rodzajów pamięci

Wybrany Element i uzasadnienie:

Moduł Czytnika Kart MicroSD. System ma zapisywać informacje w logu, który może zostać wyeksportowany, Karta MicroSD to najprostszy sposób, aby użytkownik końcowy mógł fizycznie wyjąć nośnik i przenieść logi. Dodatkowo zapewnia dużą pojemność. W celu obsługi komunikacji z kartą SD – wybrałem moduł czytnika kart pamięci SD dostępny w serwisie Botland.com.pl (MOD-08230) [Link](#).

Rodzaj danych	Miejsce przechowania	Uzasadnienie
Szablony linii papilarnych	Wewnętrzna pamięć R307	Moduł R307 ma własną pamięć Flash na 1000 szablonów, co jest najbezpieczniejsze i najszybsze do weryfikacji. Nie ma potrzeby przenoszenia ich na kartę SD.
Logi Czasu Pracy/Nauki	Karta MicroSD	Logi (np. ID:101, Tryb: Praca, Wejście: 2025-10-28 09:00:00) są głównym celem karty SD. Wymóg eksportu logów jest najlepiej spełniony przez fizycznie wymienną kartę SD.
Konfiguracja Systemu	Wewnętrzna pamięć Flash ESP32	Główne ustawienia (np. adres sieciowy, bieżący tryb) przechowuje się w małej części wewnętrznej pamięci Flash mikrokontrolera) dla szybkiego dostępu. Konfiguracje obszerne lub te, które chce zmieniać administrator bez przeprogramowania (np. limit czasu Odpoczynku) mogą być przechowywane w pliku konfiguracyjnym (np. .ini lub .json) na karcie MicroSD.

Tab 6. Działanie pamięci w systemie

Wniosek: Karta SD będzie używana głównie do Logów i opcjonalnie do dużych lub modyfikowalnych plików konfiguracyjnych, ale nie do szablonów biometrycznych.

Sygnalizacja/zmiana trybu

System będzie posiadał dodatkową sygnalizację w postaci diody statusu, potwierdzenia operacji (np. pomyślna weryfikacja). Sygnalizacja wizualna oraz dźwiękowa.

Element	Wymagane I/O	Napięcie	Uwagi
Dioda Led	1 linia I/O GPIO	3.3 V	Dioda LED (np. zielona dla sukcesu, czerwona dla błędu) wymaga podłączenia przez rezystor ograniczający prąd do pinu GPIO ESP32. Zużywa minimalną ilość prądu.

Tab 7. Dodatkowa sygnalizacja wizualna

Rozwiązanie	Wymagane I/O	Wymagania Prądowe/Napięciowe
Buzzer Pasywny	1 linia I/O (GPIO zdolny do generowania PWM)	Niskie. Idealny do generowania różnych tonów (np. inny dźwięk dla błędu i inny dla sukcesu).
Buzzer Aktywny	1 linia I/O (GPIO)	Może wymagać większego prądu, często wymaga tranzystora do sterowania. Generuje tylko jeden ton.

Tab 8. Dodatkowa sygnalizacja dźwiękowa - porównanie

Wybieram Buzzer Pasywny, ponieważ pozwala to na generowanie różnych dźwięków i melodii (np. trzykrotny sygnał błędu, pojedynczy sukces). Jest to bardziej informacyjne i elastyczne niż jeden ton buzzera aktywnego. Buzzer pasywny wymaga 1 linii GPIO (z obsługą PWM). Przykład: MOD-02963 [Link](#).

Ostatnim elementem (wej/wyj) będzie urządzenie – pozwalające użytkownikowi na zmianę trybu pracy na urządzeniu – enkoder obrotowy.

Enkoder obrotowy pozwala użytkownikowi kręcić (zmiana trybu) oraz naciskać (potwierdzenie wyboru).

Moduł Enkodera	Napięcie zasilania	Napięcie logiczne dla sygnałów	Cena
Waveshare 9533	Od 3V do 5V	3V do 5V	14,90zł
OkyStar KY-040	5V	5V	7,90zł
Iduino SE055	5V	5V	11,90zł

Tab 9. Porównanie enkoderów

Wybrany Element: Czujnik obrotu, enkoder z przyciskiem - Moduł Waveshare 9533. Moduł ten jest jedynym, który bezpośrednio pracuje z napięciem 3V do 5V, co potencjalnie pozwala uniknąć konieczności dodawania konwertera poziomów logicznych.

Wybór mikrokontrolera

Wymagania:

- **13 linii I/O** (min.)
- **Interfejsy komunikacyjne:** 1x UART, 1x I2C, 1x SPI
- **Wyjścia:** 1x PWM (dla buzzera), 3x Wejścia z obsługą przerwań (dla enkodera)
- **Wbudowana Pamięć:** Wystarczająca Flash i RAM do obsługi złożonej logiki, protokołu UART (R307), systemu plików (MicroSD FatFS) i bibliotek graficznych (OLED).
- **Łatwość Prototypowania:** biblioteki, wsparcie społeczności...

Na podstawie złożoności projektu i konieczności obsługi zaawansowanych protokołów (TCP/IP dla ewentualnej chmury/eksportu logów, FatFS dla karty SD, grafiki), najlepszymi kandydatami są nowoczesne, wydajne rodziny mikrokontrolerów.

Rodzina	Architektura	Zalety dla projektu	Wady/Uwagi
STM32	ARM Cortex - M	Wysoka wydajność, wiele sprzętowych interfejsów (UART, I2C, SPI), bardzo rozbudowana ekosystem i narzędzia.	Bardziej stroma krzywa uczenia się, mniej gotowych modułów "plug-and-play" z Wi-Fi.
ESP32	Xtensa LX6/7	Wbudowane Wi-Fi/Bluetooth (idealne dla zapisu danych w chmurze), duża ilość pamięci RAM/Flash, łatwość programowania (Arduino/ESP-IDF).	Wyższy pobór mocy niż STM32, ogromna ilość projektów z wykorzystaniem ESP32, spore wsparcie bibliotek i społeczności. Dużo gotowych modułów.
ATMega	AVR	Łatwe w użyciu	Zbyt mała wydajność i pamięć do obsługi systemów plików, bibliotek graficznych i protokołów TCP/IP. Brak wbudowanego Wi-Fi.

Tab 10. Porównanie rodzin mikrokontrolerów.

Projekt zakłada możliwość zapisu danych w chmurze. Wbudowany moduł Wi-Fi i Bluetooth w ESP32 sprawia, że jest to najbardziej logiczny i efektywny kosztowo wybór, eliminując potrzebę zewnętrznego modułu komunikacyjnego Ethernet/GSM. Dodatkowo 2-rdzeniowy procesor i ilość pamięci Flash/RAM wystarczy do obsługi:

- Jednoczesną obsługę systemu plików **FatFS** (karta SD) i bibliotek graficznych (OLED).
- Komunikację po **UART** (R307), **SPI** (SD Card) i **I2C** (OLED/RTC).
- Złożoną logikę trybów pracy/nauki/odpoczynku.

No i łatwość prototypowania, dużo bibliotek i materiałów dostępnych w Internecie.

W obrębie rodziny ESP32 istnieje kilka popularnych modułów. Wybrałem moduł, który jest łatwo dostępny, ma wyprowadzoną wystarczającą liczbę pinów i stabilne zasilanie.

Moduł	Chip	Liczba pinów	Pamięć Flash	Uwagi
ESP32 DevKitC	ESP32-WROOM-32	30	4MB	Najpopularniejsza płytki, stabilne zasilanie/USB-UART, wystarczająca liczba GPIO.
ESP32-CAM	ESP32-S	≈ 15	4MB	Zbyt mało dodatkowych pinów GPIO
ESP32-S2/S3	ESP32-S2/S3	30 - 40	4MB-16MB	Nowsze wersje, większa wydajność i więcej GPIO.

Tab 10. Porównanie mikrokontrolerów.

Ostatecznie wybieram ESP32 DevKitC. Moduł ESP32 DevKitC wyprowadza 30 pinów GPIO, co jest ponad dwukrotnie więcej niż 13 wymaganych linii I/O. Zapewnia to elastyczność w podłączaniu peryferii i rezerwę.

Oferuje liczne sprzętowe porty: 3x UART, 2x I2C, 2x SPI (VSPI i HSPI), co z łatwością obsłuży wszystkie wybrane peryferia bez konieczności multipleksowania.

Zasilanie systemu

Zaprojektowanie systemu zasilania musi uwzględniać dwa główne poziomy napięć i jeden element konwersji logicznej.

- **Główne Źródło:** Zasilacz sieciowy USB (prawdopodobnie 5V, np. z ładowarki telefonu).
- **Poziomy Napięcie Wymagane:**
 - 5V (Zasilanie): Dla czytnika R307 oraz buzzera
 - 3.3V (Logika/MCU): Dla ESP32, OLED, RTC, MicroSD i wszystkich linii logicznych.
 - Podtrzymanie: Bateria pastylkowa dla RTC (już zapewnione przez moduł DS1307).

Linia zasilania	Elementy zasilane	Sposób realizacji	Uwagi
VIN (5V)	R307, Buzzer (cewka), Gniazdo USB*	Bezpośrednio z zasilacza sieciowego USB.	*USB na płytce dewelopperskiej
3.3V	ESP32 MCU, OLED, RTC, MicroSD, Enkoder, LED	Wbudowany stabilizator LDO (na płytce ESP32 DevKitC.	DevKitC ma już stabilizator 5V do 3.3V zasilający MCU i peryferia 3.3V
Podtrzymanie RTC	DS1307 V-BAT	Bateria pastylkowa CR2032	Niezależne zasilanie, odseparowane od VIN 3.3V

Tab 11. Linie zasilania.

Dodatkowo dla bezpieczeństwa komunikacji będzie konieczne zastosowanie konwertera poziomów logicznych (level shifter):

Wymagana konwersja	Układ I/O	Wymagane Linie	Wybrany moduł
5V → 3.3V	R307 TXD → ESP32 RXD	1 linia (TX R307)	Dwukanałowy Level Shifter Iduino ST1167

Tab 12. Konwerter napięcia

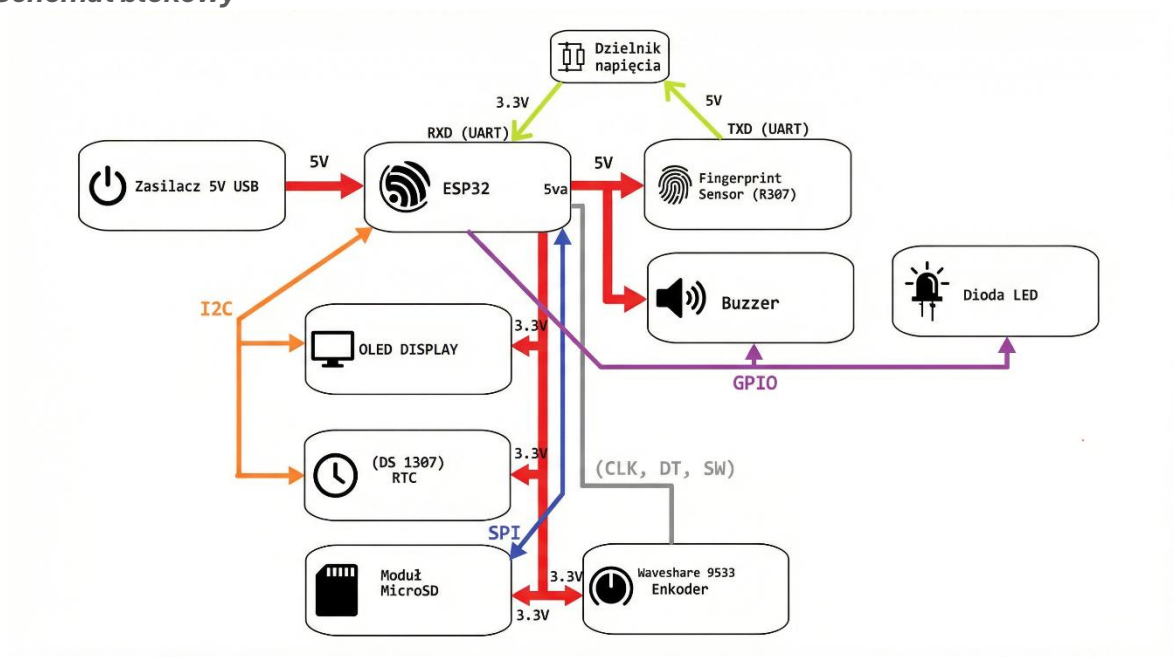
Wybrałem gotowy, mały moduł konwertera logicznego. Potrzebuję tylko jednego kierunku konwersji 5V → 3.3V na linii TXD czytnika R307 do pinu RXD ESP32.

Przyjmijmy zasilacz sieciowy USB o wydajności co najmniej 1A (5W).
Szacunkowy pobór prądu:

Element	Napięcie	Max pobór prądu (szacunkowo)	Uwagi
ESP32 (Wi-Fi On)	3.3V	≈ 200mA – 300mA	Największy konsument prądu.
Czytnik R307 (Skanowanie)	5V	≈ 150mA	Chwilowy pik podczas skanowania.
Wyświetlacz OLED	3.3V	≈ 20mA	Mały pobór.
MicroSD (Zapis)	3.3V	≈ 50mA	Mały pobór.
Buzzer	5V	≈ 30mA	Chwilowy pobór
Całkowity Pobór (W szczycie)		≈ 500mA	Zasilacz 1A jest wystarczający i zapewnia bezpieczny margines.

Tab 13. Szacunkowy pobór mocy

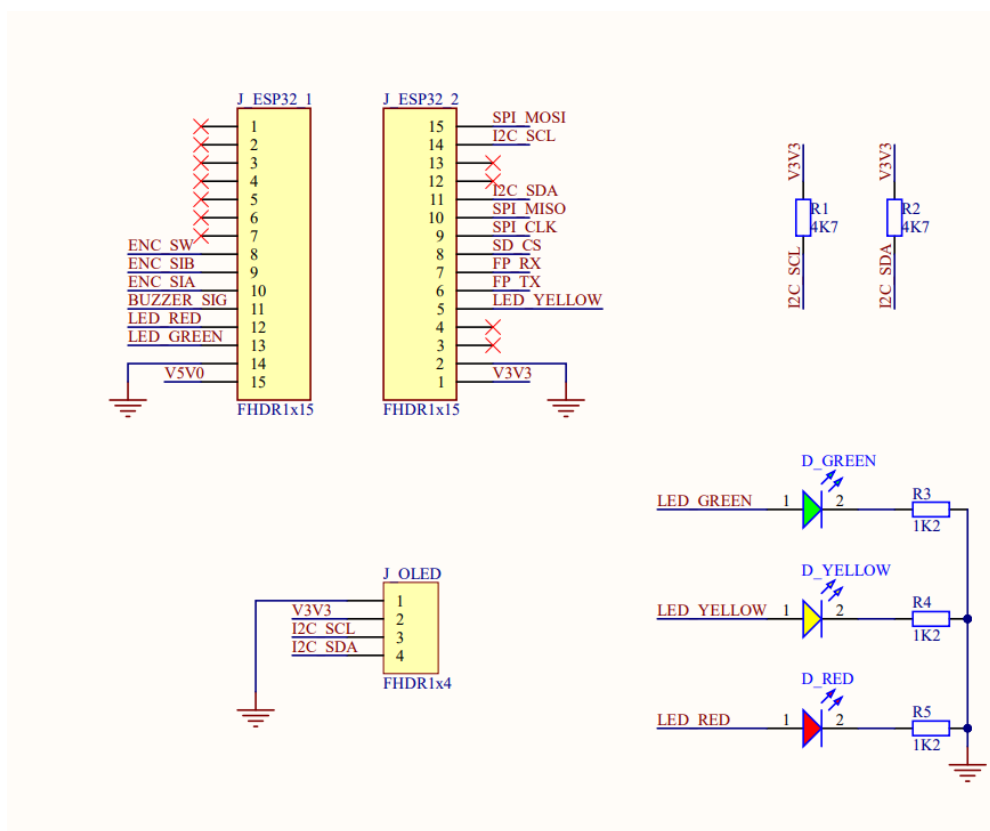
Schemat blokowy



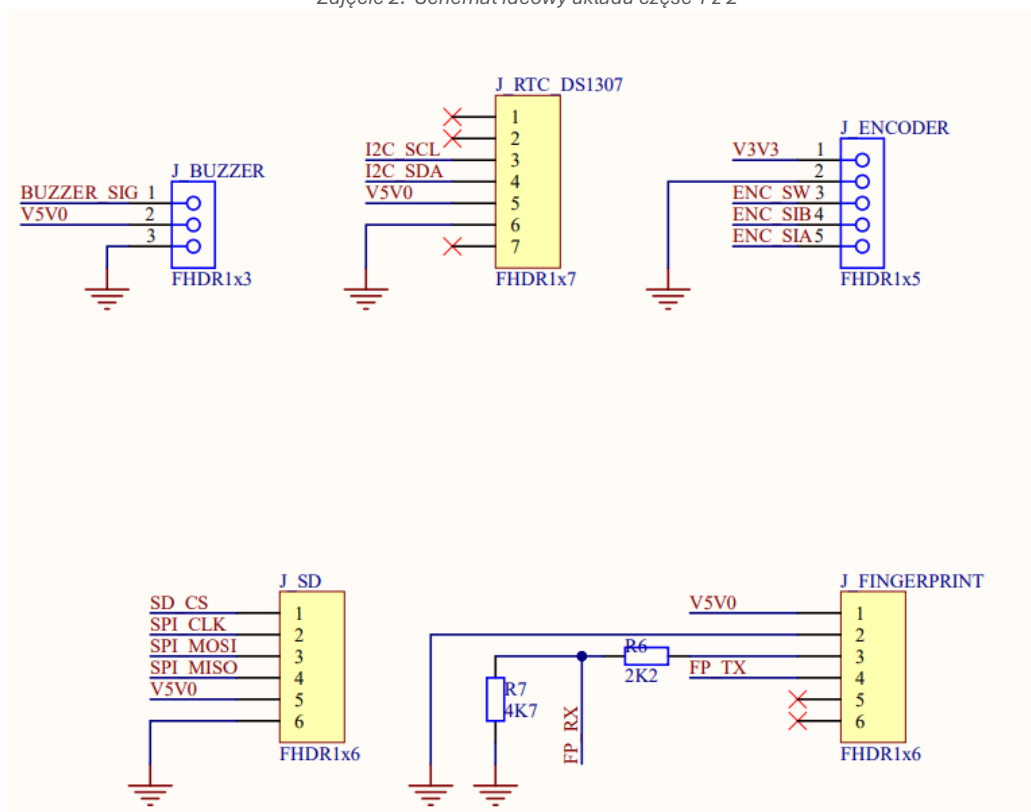
Zdjęcie 1. Schemat blokowy układu

Specyfikacja wewnętrzna urządzenia

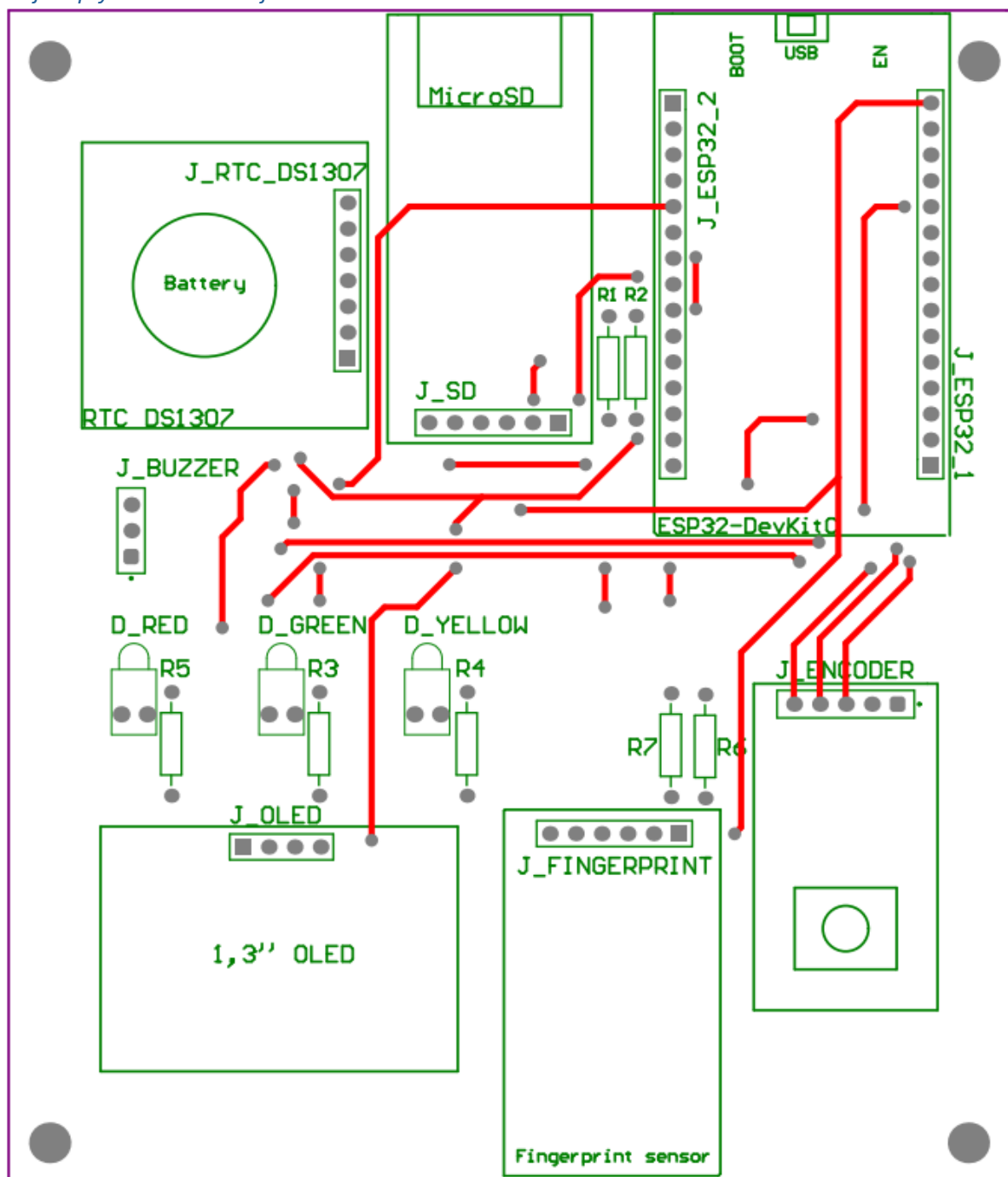
Schemat ideowy urządzenia



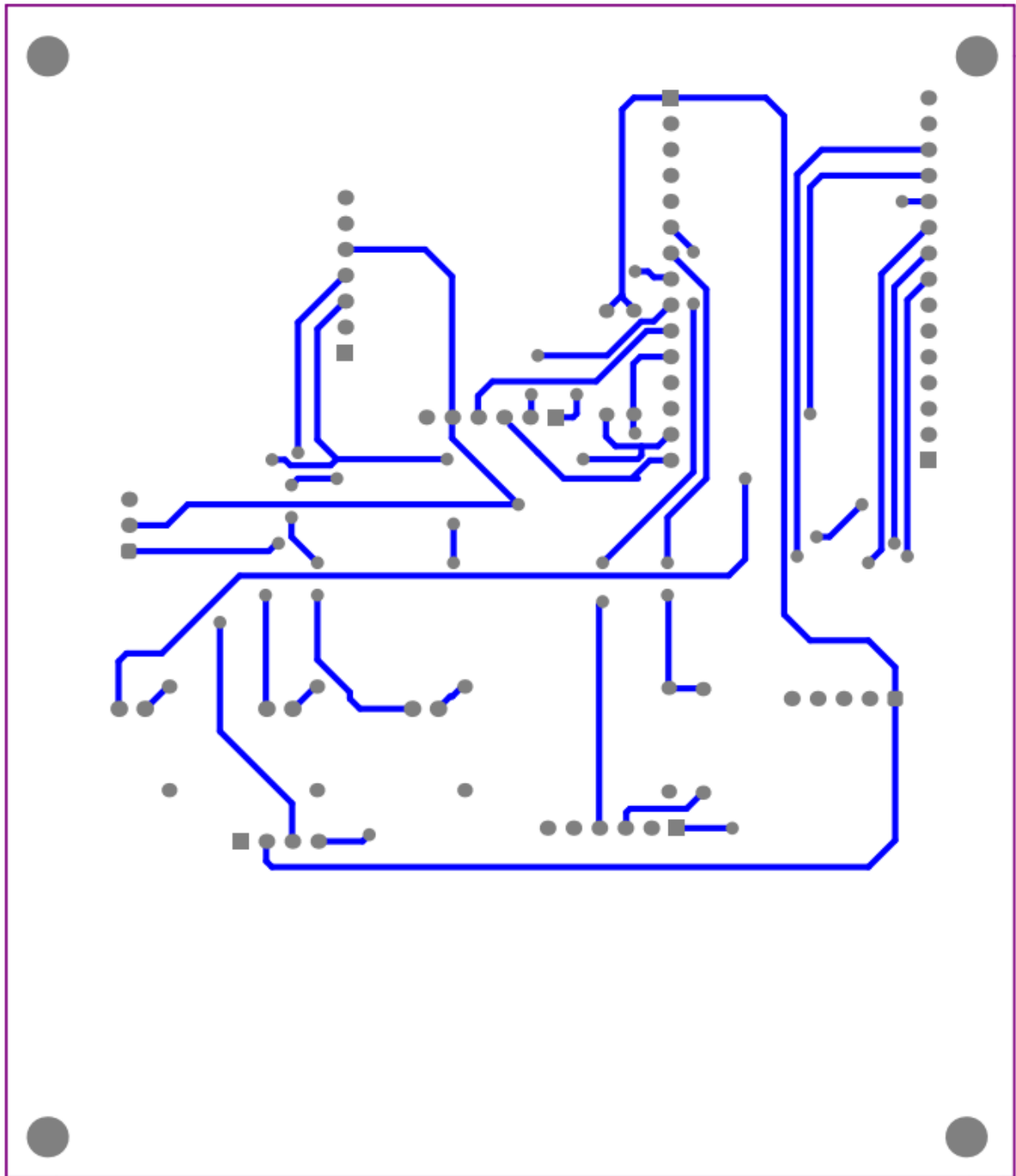
Zdjęcie 2. Schemat ideowy układu część 1 z 2



Zdjęcie 3. Schemat ideowy układu część 2 z 2

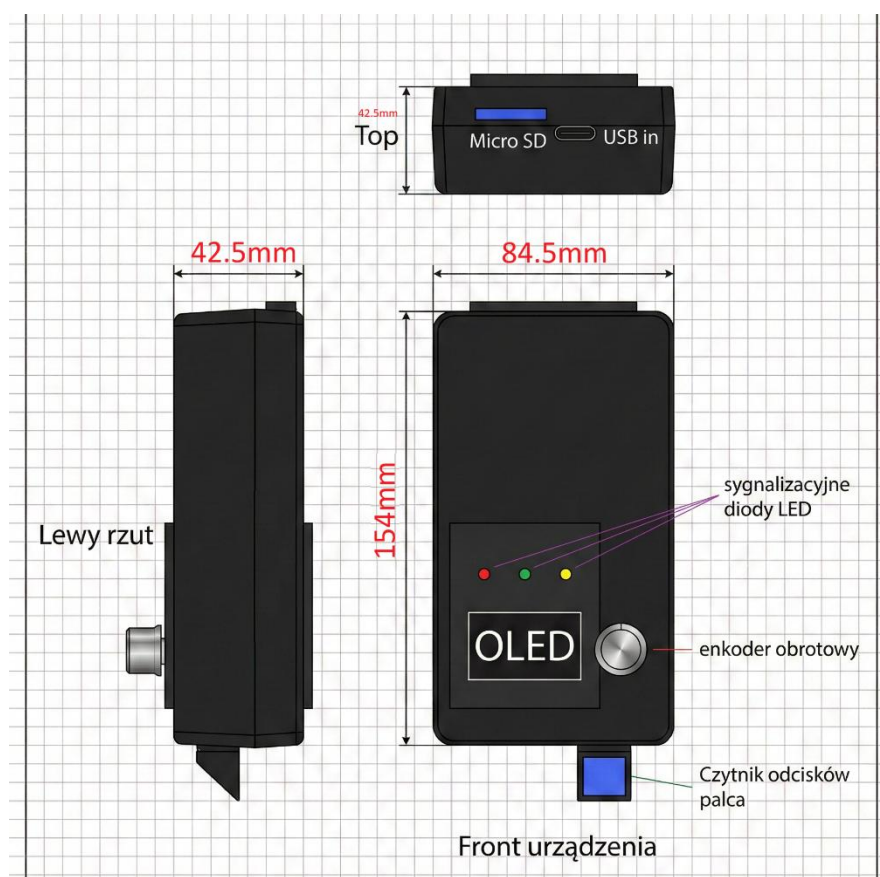


Zdjęcie 4. Projekt płytki drukowanej – TOP Layer

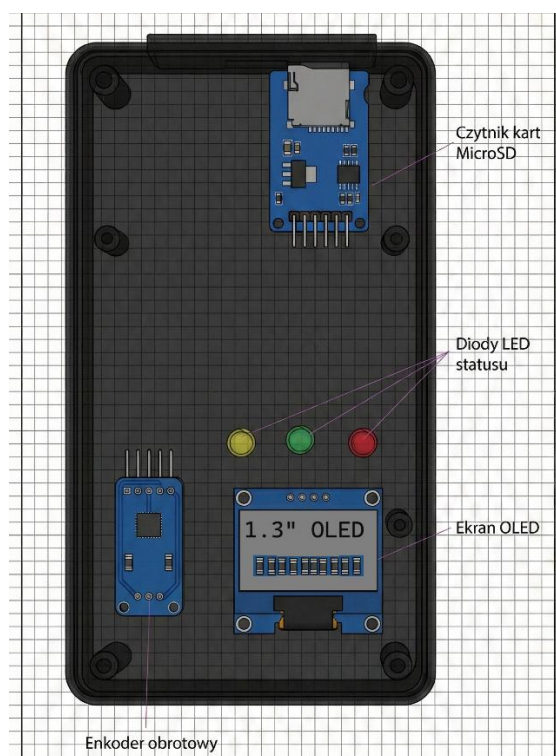


Zdjęcie 4. Projekt płytki drukowanej – Bottom Layer

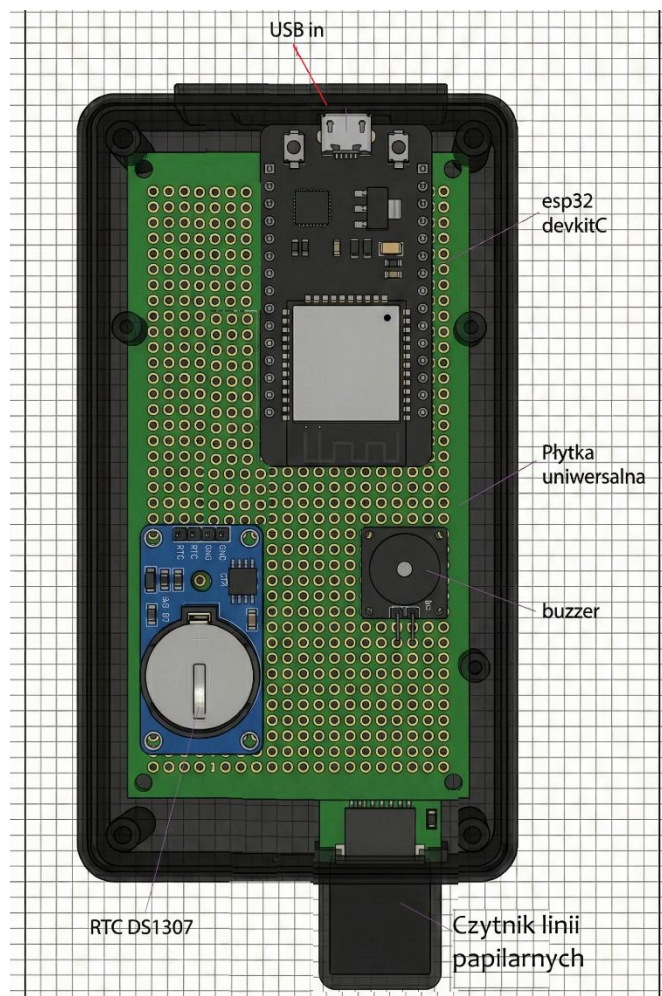
Rozmieszczenie elementów w obudowie



Zdjęcie 5. Schemat koncepcyjny urządzenia



Zdjęcie 6. Rozmieszczenie elementów w obudowie (frontowa część – widok od środka)



Zdjęcie 7. Rozmieszczenie elementów w obudowie (spodnia część)



Zdjęcie 8. Rzeczywisty układ front



Zdjęcie 9. Rzeczywisty układ od boku



Zdjęcie 9. Rzeczywisty układ – wejście zasilania USB oraz wejście na kartę MicroSD

Mikrokontroler ESP32-DevKitC (ESP32-WROOM-32) – Centrum całego systemu. Odpowiada za koordynację prac wszystkich peryferiów. Inicjalizuje magistrale komunikacyjne (I2C, SPI, UART), przetwarza dane z czytnika biometrycznego, obsługuje logikę biznesową (zliczanie czasu, zmiana trybów Praca/Nauka) oraz zarządza zapisem danych na kartę SD i wyświetlaniem treści na ekranie OLED.

Czytnik Linii Papilarnych (R307) – Moduł biometryczny wyposażony we własny procesor DSP. Odpowiada za akwizycję obrazu odcisku palca, tworzenie jego cyfrowego szablonu oraz porównywanie go z bazą zapisaną w swojej wewnętrznej pamięci Flash. Komunikuje się z mikrokontrolerem przez interfejs UART, zwracając jedynie wynik weryfikacji (ID użytkownika lub błąd).

Wyświetlacz OLED 1.3" (SH1106) – Element interfejsu użytkownika (HMI). Prezentuje aktualną datę i godzinę, wybrany tryb pracy (Praca/Nauka/Odpoczynek) oraz komunikaty statusowe (np. "Przytóż palec", "Zalogowano", "Błąd").

Zegar Czasu Rzeczywistego (RTC DS1307) – Układ odpowiedzialny za niezależne od zasilania sieciowego odliczanie czasu. Posiada własne podtrzymanie bateryjne (bateria CR2032), dzięki czemu system po ponownym uruchomieniu zna aktualną datę i godzinę, co jest krytyczne dla logowania czasu pracy.

Moduł Czytnika Kart MicroSD – Pełni funkcję pamięci masowej dla logów systemowych. Zapisuje zdarzenia (wejścia/wyjścia) w plikach (.csv), co umożliwia późniejszy eksport danych do komputera PC.

Enkoder Impulsowy (Obrotowy) – Służy do nawigacji po menu urządzenia. Umożliwia zmianę aktualnego trybu pracy (obróć) oraz zatwierdzanie wyboru (wciśnięcie przycisku).

Sygnalizacja wizualno-dźwiękowa (LED + Buzzer) – Zespół elementów wykonawczych informujących o statusie operacji w sposób natychmiastowy (zielona dioda/krótki sygnał = sukces; czerwona dioda/długi sygnał = błąd).

Algorytm oprogramowania (można powiększyć) – opis słowny poniżej

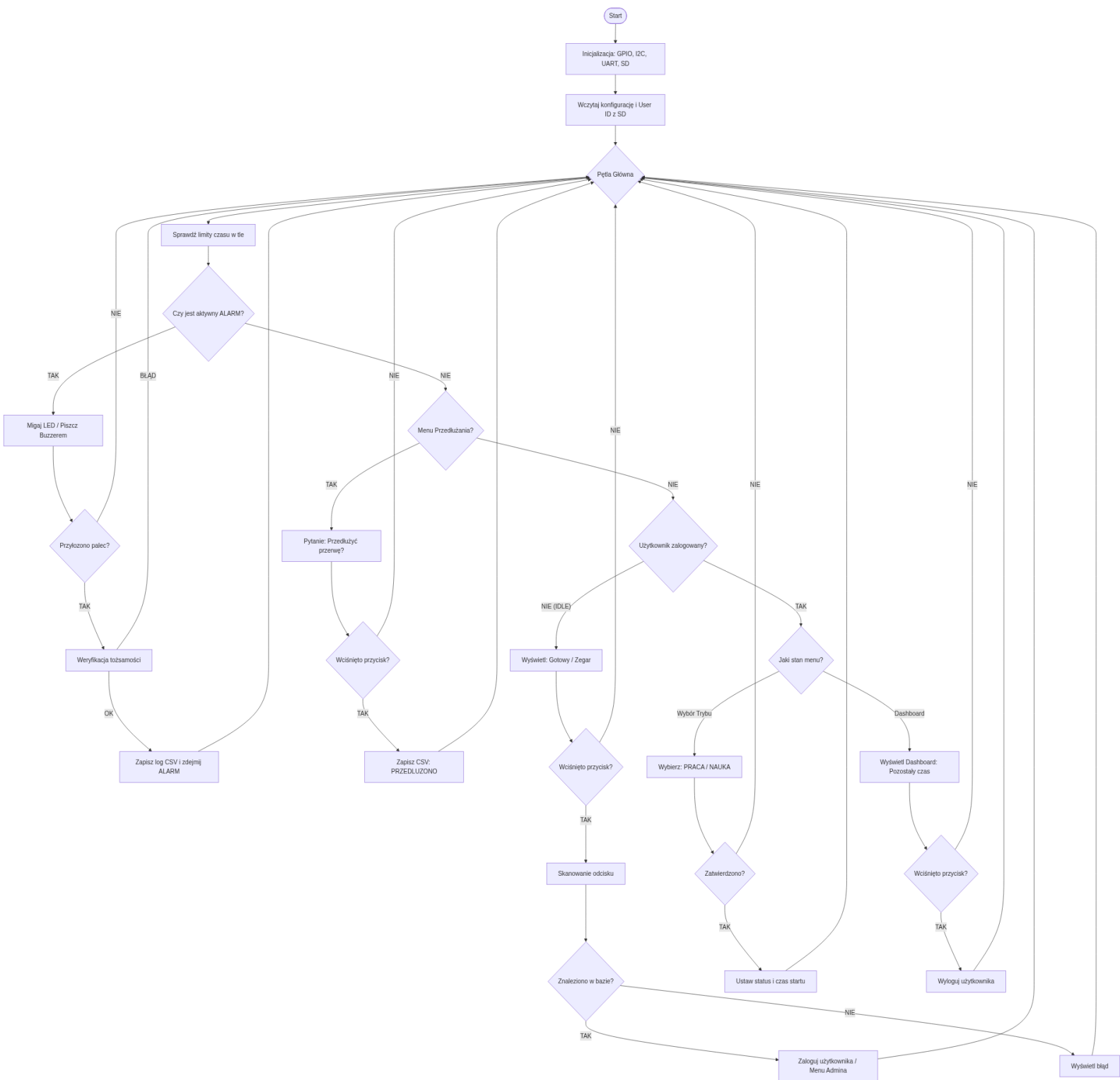


Diagram 1. Oprogramowanie układu

Algorytm sterujący pracą urządzenia (opis słowny):

1. **Inicjalizacja:** Po uruchomieniu mikrokontroler konfiguruje piny GPIO, magistrale komunikacyjne (I2C dla OLED/RTC, UART dla czytnika, SPI dla SD) oraz wczytuje plik konfiguracyjny `config.txt`.
2. **Pętla Główna:** Program wchodzi w nieskończoną pętlę, w której w pierwszej kolejności sprawdzane są **timery tła** (funkcja `check_background_timers`). Jeśli czas pracy/przerwy użytkownika przekroczy limit zdefiniowany w konfiguracji, ustawiana jest flaga ALARMU.
3. **Obsługa Alarmu (Priorytet):** Jeśli flaga alarmu jest aktywna, urządzenie blokuje inne funkcje, uruchamia sygnalizację świetlną i dźwiękową oraz oczekuje na weryfikację biometryczną w celu potwierdzenia obecności użytkownika.
4. **Stan IDLE (Czuwanie):** Gdy nikt nie jest zalogowany, system wyświetla aktualną godzinę. Wciśnięcie enkodera wybudza czytnik linii papilarnych. Po poprawnej weryfikacji następuje logowanie użytkownika.
5. **Stan Pracy/Nauki:** Zalogowany użytkownik widzi panel główny z licznikiem pozostałego czasu. Może wylogować się przyciskiem lub system automatycznie przerwie pracę po osiągnięciu limitu (wówczas nastąpi przejście do Alarmu).
6. **Zapis danych:** Każda kluczowa zmiana stanu (Logowanie, Start Pracy, Koniec Przerwy, Alarm) generuje wpis w pliku `data.csv` na karcie SD zawierający datę, ID użytkownika i typ zdarzenia.

Opis wszystkich ważniejszych zmiennych

users – Tablica struktur typu `UserSession` przechowująca dane o sesjach wszystkich użytkowników (do 10 osób – możliwością rozszerzenia). Każdy element zawiera ID użytkownika, jego aktualny status (np. PRACA, PRZERWA), czasy rozpoczęcia aktywności oraz liczniki (ilość przerw, przepracowane minuty).

config – Struktura globalna przechowująca parametry konfiguracyjne systemu wczytane z karty SD, takie jak: limity czasu pracy (`work_limit`), nauki (`study_limit`), przerw (`break_limit`) oraz maksymalną liczbę przedłużeń przerwy.

u8g2 – Struktura handlera biblioteki graficznej, wykorzystywana do komunikacji z wyświetlaczem OLED poprzez interfejs I2C oraz przechowywania bufora obrazu.

button_flag – Flaga typu `volatile bool` ustawiana w procedurze obsługi przerwania (ISR) w momencie naciśnięcia przycisku enkodera. Informuje pętlę główną o konieczności obsłużenia akcji zatwierdzenia.

pending_alert_id – Zmienna przechowująca identyfikator użytkownika, dla którego wykryto przekroczenie limitu czasu (`alarm`). Wartość -1 oznacza brak aktywnego alarmu.

menu_step – Zmienna sterująca maszyną stanów interfejsu użytkownika. Określa, jaki ekran jest aktualnie wyświetlany (np. 0 – ekran główny, 50 – menu administratora, 150 – dashboard użytkownika).

next_enroll_id – Zmienna przechowująca kolejny wolny numer ID dla nowo rejestrowanego użytkownika. Jej wartość jest odczytywana i zapisywana w pliku `last_id.txt` na karcie SD.

current_user_id – Przechowuje ID aktualnie zalogowanego użytkownika obsługującego menu. Służy do kontekstowego wyświetlania opcji (np. Start Pracy / Wyloguj).

Opis wszystkich procedur

void app_main(void) – Główna funkcja programu zawierająca pętlę nieskończoną. Odpowiada za inicjalizację peryferiów (GPIO, I2C, UART, SD), obsługę maszyny stanów interfejsu (wyświetlanie menu) oraz priorytetową obsługę alarmów.

void check_background_timers() – Funkcja wywoływana w każdej iteracji pętli głównej. Przeszukuje tablicę `users`, oblicza upływ czasu dla aktywnych sesji i porównuje go z limitami w strukturze `config`. W razie przekroczenia ustawia odpowiedni status użytkownika i flagę alarmu.

int scan_blocking() – Procedura blokująca, która komunikuje się z czytnikiem linii papilarnych R307 przez UART. Oczekuje na przyłożenie palca, wykonuje skanowanie i przeszukanie bazy. Zwraca ID rozpoznanego użytkownika lub kod błędu.

void enroll_blocking() – Procedura realizująca proces rejestracji nowego użytkownika (tryb administratora). Prowadzi użytkownika przez proces dwukrotnego przyłożenia palca, generuje szablon biometryczny i zapisuje go w pamięci czytnika pod nowym ID.

void load_config() – Odczytuje plik `config.txt` z karty pamięci SD, parsuje zawartość i aktualizuje strukturę `config`. Obsługuje również jednorazową synchronizację czasu RTC, jeśli w pliku ustawiono flagę `set_time=1`.

void write_csv(int id, const char *act, int dur, int lim, const char *stat) – Funkcja logująca zdarzenia systemowe. Pobiera aktualną datę z RTC i dopisuje sformatowany wiersz (CSV) do pliku `data.csv` na karcie SD.

void draw_ui(const char* t, const char* c, const char* b) – Funkcja warstwy abstrakcji wyświetlacza. Czyści bufor, rysuje nagłówek z zegarem oraz treść (tytuł i opcje menu) przekazaną w argumentach, a następnie wysyła bufor do ekranu OLED.

void draw_dashboard(UserSession* u, int remaining_min) – Specjalizowana funkcja wyświetlająca panel użytkownika z informacjami o pozostałym czasie pracy/nauki oraz statystykami przerw.

void send_cmd(...) / uint8_t get_resp(...) – Niskopoziomowe funkcje obsługujące protokół komunikacyjny czytnika R307. Formułują pakiety danych z sumą kontrolną i obsługują transmisję przez UART.

void set_rtc_time(...) / **time_t get_seconds()** – Funkcje obsługujące moduł RTC DS1307. Konwertują format daty z BCD na dziesiętny (i odwrotnie) oraz komunikują się z układem przez magistralę I2C.

void IRAM_ATTR isr_enc() / **isr_btn()** – Procedury obsługi przerwań dla enkodera i przycisku. Aktualizują zmienne globalne **encoder_pos** oraz **button_flag** w czasie rzeczywistym, bez blokowania procesora.

Opis interakcji oprogramowania z układem elektronicznym

Oprogramowanie steruje układem elektronicznym wykorzystując zarówno cyfrowe magistrale komunikacyjne, jak i bezpośrednie sterowanie liniami GPIO oraz obsługę przerwań.

Komunikacja z czytnikiem biometrycznym (R307):

Odbywa się poprzez sprzętowy interfejs **UART2** (piny **FP_TX_PIN** / **FP_RX_PIN**). Oprogramowanie wysyła pakiety danych zgodnie z protokołem producenta, a następnie oczekuje na odpowiedź (potwierdzenie wykonania, ID użytkownika). Funkcje **uart_write_bytes** oraz **uart_read_bytes** realizują fizyczną transmisję bitów.

Obsługa wyświetlacza OLED:

Wykorzystuje bibliotekę **u8g2**¹, która komunikuje się ze sterownikiem ekranu (SH1106) poprzez magistralę **I2C** (piny **I2C_SDA**, **I2C_SCL**). Oprogramowanie przygotowuje bufor obrazu w pamięci RAM mikrokontrolera, a funkcja **u8g2_SendBuffer** przesyła go szeregowo do wyświetlacza w celu odświeżenia treści.

Odczyt czasu (RTC DS1307):

Mikrokontroler inicjuje transmisję **I2C** do adresu **0x68**. Funkcja **i2c_master_write_read_device** wysyła adres rejestru (np. **0x00** dla sekund), a następnie odczytuje 7 bajtów danych, które są programowo konwertowane z formatu BCD na format dziesiętny zrozumiały dla człowieka.

Zapis danych (Karta MicroSD):

Wykorzystuje interfejs **SPI** (PIN_MOSI, PIN_MISO, PIN_CLK, PIN_CS). System plików FAT (obsługiwany przez **esp_vfs_fat**) mapuje wywołania funkcji wysokiego poziomu (jak **fopen**, **fprintf**) na niskopoziomowe komendy SPI wysyłane do karty pamięci.

Interfejs wejściowy (Enkoder):

Zmiana stanu na pinach enkodera (**ENC_SIA**, **ENC_SIB**) oraz przycisku (**ENC_SW**) wyzwała sprzętowe przerwy (GPIO ISR). Funkcje obsługi przerwań (**isr_enc**, **isr_btn**) natychmiastowo aktualizują zmienne globalne (flagi), co pozwala na płynną obsługę menu bez blokowania pętli głównej.

Sterowanie wyjściami (LED/Buzzer): Diody LED oraz buzzer są sterowane bezpośrednio poprzez zmianę stanów logicznych na pinach GPIO za pomocą funkcji **gpio_set_level**. Buzzer jest kluczowany programowo w pętli (funkcja **beep**), co pozwala na generowanie sygnałów akustycznych o określonej częstotliwości i czasie trwania.

¹ <https://github.com/olikraus/u8g2>

Szczegółowy opis działania ważniejszych procedur

1. scan_blocking() – **Procedura skanowania odcisku palca** Funkcja ta implementuje maszynę stanów oczekującą na przyłożenie i weryfikację palca. Działanie opiera się na sekwencyjnym wysyłaniu komend do modułu R307:

- W pętli (z ograniczeniem do 50 prób) wysyłana jest komenda **GetImage** (0x01). Funkcja sprawdza odpowiedź czujnika – jeśli kod powrotu wynosi 0x00, oznacza to, że palec został poprawnie wykryty.
- Następnie wysyłana jest komenda **GenChar** (0x02), która przetwarza obraz palca na postać cyfrową (szablon) w buforze czujnika.
- Kluczowym krokiem jest komenda **Search** (0x04), która zleca procesorowi DSP czytnika porównanie utworzonego szablonu z całą bazą zapisaną w pamięci flash modułu.
- Funkcja zwraca identyfikator użytkownika (**PageID**) w przypadku sukcesu lub kod błędu, jeśli nie znaleziono dopasowania.

2. check_background_timers() – **Weryfikacja limitów czasu** Jest to krytyczna procedura odpowiedzialna za logikę RCP, wywoływana w każdym obiegu pętli głównej (pod warunkiem, że nie ma aktywnego alarmu).

- Funkcja iteruje przez tablicę wszystkich użytkowników (**users**). Dla każdego zalogowanego użytkownika (status PRACA, NAUKA lub PRZERWA) pobiera aktualny czas z RTC (w sekundach).
- Oblicza czas trwania bieżącej sesji jako różnicę: `czas_teraz - czas_startu`.
- Następuje porównanie obliczonego czasu z limitami zdefiniowanymi w strukturze **config** (np. `work_limit`).
- Jeśli limit zostanie przekroczony, funkcja zmienia status użytkownika na status alarmowy (np. `STATUS_ALERT_WORK`) i ustawia zmienną globalną **pending_alert_id**. To wymusza na pętli głównej natychmiastowe przejście do obsługi alarmu (zablokowanie interfejsu i uruchomienie sygnalizacji).

3. app_main() – **Główna pętla sterująca** Implementuje nadrzędną logikę urządzenia w oparciu o priorytety zdarzeń:

- **Priorytet 1 (Alarmy):** Sprawdza zmienną `pending_alert_id`. Jeśli jest ustawiona, steruje miganiem diody i buzzerem. Oczekuje na weryfikację biometryczną. Po poprawnym skanie zmienia stan użytkownika (np. z PRACY na PRZERWĘ) i loguje zdarzenie.
- **Priorytet 2 (Menu Przedłużania):** Jeśli aktywny jest `menu_step == 100`, obsługuje logikę przedłużania przerwy (zapis na kartę SD, reset timerów).
- **Priorytet 3 (Stan IDLE):** Jeśli nikt nie jest zalogowany, wyświetla ekran główny. Obsługuje wejście w tryb administratora (długie kręcenie enkoderem) lub logowanie użytkownika (krótkie wciśnięcie i skan).
- **Priorytet 4 (Menu Użytkownika):** Obsługuje wybór trybu (Praca/Nauka) oraz wyświetlanie panelu ze statystykami, reagując na sygnały z enkodera zaktualizowane w przerwaniach.

4. load_config() – Parser konfiguracji Funkcja ta zapewnia elastyczność systemu bez konieczności rekompilacji kodu.

- Otwiera plik **config.txt** z karty SD.
- Czyta plik linia po linii, wyszukując słowa kluczowe (np. **work=**, **break=**).
- Konwertuje wartości tekstowe na liczby całkowite (atoi) i przypisuje je do globalnej struktury **config**.
- Unikalną cechą jest obsługa parametru **set_time=1**. Jeśli zostanie wykryty, funkcja parsuje datę z pliku, ustawia zegar RTC sprzętowo, a następnie nadpisuje plik konfiguracyjny, zmieniając **set_time** na 0, aby zapobiec ponownemu przestawieniu zegara przy kolejnym restarcie.

Specyfikacja zewnętrzna urządzenia:

Opis funkcji elementów sterujących urządzeniem

Urządzenie posiada dwa główne interfejsy wejściowe umożliwiające interakcję użytkownika z systemem:

Enkoder obrotowy z przyciskiem – Jest to główny element nawigacyjny.

- **Obrót:** Pozwala na przełączanie się pomiędzy opcjami w menu (np. wybór między trybem "PRACA" a "NAUKA", nawigacja w menu administratora).
- **Wciśnięcie przycisku:** Służy do zatwierdzania wyboru, wybudzania urządzenia ze stanu czuwania oraz inicjowania procesu skanowania odcisku palca.

Czytnik linii papilarnych (Sensor optyczny R307) – Pełni funkcję sensora biometrycznego.

Służy do wprowadzania danych (obrazu odcisku palca) w celu weryfikacji tożsamości użytkownika podczas logowania oraz rejestracji nowych osób w systemie.

Opis funkcji elementów wykonawczych

Wyświetlacz graficzny OLED 1.3" – Prezentuje aktualny stan systemu, datę i godzinę, menu wyboru trybów oraz komunikaty tekstowe (np. "Witaj", "Błąd", "Koniec Czasu"). Stanowi główny interfejs sprzężenia zwrotnego (HMI).

Diody sygnalizacyjne LED – Zapewniają natychmiastową informację o statusie operacji:

- **Zielona:** Sygnalizuje pomyślną weryfikację użytkownika lub poprawnie wykonaną operację.
- **Czerwona:** Sygnalizuje błąd (np. brak odcisku w bazie) lub odmowę dostępu.
- **Żółta:** Sygnalizuje stan "PRZERWA" lub ostrzeżenie (np. zbliżający się koniec limitu czasu).

Buzzer – Generuje sygnały akustyczne:

- **Krótki sygnał:** Potwierdzenie kliknięcia lub sukcesu.
- **Seria sygnałów:** Alarm (przekroczenie limitu czasu pracy/przerwy) lub błąd krytyczny.

Opis reakcji oprogramowania na zdarzenia zewnętrzne

Wciśnięcie przycisku enkodera (w stanie IDLE):

System wybudza się, wyświetla komunikat "Przytóż palec..." i aktywuje podświetlenie sensora biometrycznego, oczekując na weryfikację.

Przyłożenie palca do czytnika: Oprogramowanie pobiera obraz, przetwarza go i porównuje z bazą.

- W przypadku *zgodności*: Wyświetla panel użytkownika, zapala zieloną diodę i emituje krótki dźwięk.
- W przypadku *braku zgodności*: Wyświetla komunikat "**Nieznany!**", zapala czerwoną diodę i emituje sygnał błędu.

Obrót enkodera: W menu wyboru powoduje przesunięcie kursora (strzałki >) przy opcjach "PRACA" / "NAUKA".

Wykrycie przekroczenia czasu (Zdarzenie czasowe): Gdy wewnętrzny licznik przekroczy limit skonfigurowany na karcie SD, system przechodzi w stan ALARMU – blokuje normalne menu, zaczyna migać diodami i generować dźwięki do momentu potwierdzenia obecności przez użytkownika.

Skrócona instrukcja obsługi urządzenia

1. **Uruchomienie:** Podłącz urządzenie do zasilania USB. Poczekać, aż na ekranie pojawi się napis "SYSTEM RCP - Gotowy" oraz aktualna godzina.
2. **Logowanie:** Wciśnij pokrętkę enkodera. Gdy pojawi się komunikat, przytóż palec do czytnika.
3. **Wybór trybu:** Po zalogowaniu obróć pokrętkę, aby wybrać tryb "PRACA" lub "NAUKA", a następnie wciśnij pokrętkę, aby zatwierdzić.
4. **Praca:** Na ekranie widoczny jest licznik czasu pozostałego do przerwy.
5. **Przerwa/Alarm:** Gdy czas się skończy, urządzenie uruchomi alarm. Przytóż palec, aby potwierdzić przejście na przerwę.
6. **Wylogowanie:** Wciśnij przycisk na ekranie głównym, aby zakończyć sesję i wylogować się z systemu.
7. **Tryb Administratora:** (Tylko dla uprawnionych) W stanie czuwania wykonaj szybki obrót enkoderem w prawo, aby wejść w tryb dodawania nowych użytkowników. Można to również zrobić bez wykorzystania enkodera. Wówczas

należy:

- a. Pierwszy zarejestrowany odcisk palca (dowolny) będzie odciskiem Administratora.
- b. Przy następnej swojej rejestracji – administrator może wejść w swój panel i kliknąć **Dodaj Usera** wówczas nowy użytkownik powinien zeskanować swój palec 2 razy w celu weryfikacji i dodania do systemu

Opis złączy i połączeń

1. Złącza zewnętrzne:

- **Port MicroUSB (w module ESP32):** Służy do zasilania układu (5V DC) oraz programowania mikrokontrolera i diagnostyki (UART-to-USB).
- **Słot kart MicroSD:** Umożliwia montaż i demontaż karty pamięci w celu odczytu logów (data.csv) na komputerze PC.

Połączenia wewnętrzne:

- **Magistrala I2C (SDA=P21, SCL=P22):** Łączy mikrokontroler z wyświetlaczem OLED oraz zegarem RTC DS1307.
- **Magistrala SPI (MOSI=P23, MISO=P19, CLK=P18, CS=P5):** Łączy mikrokontroler z modułem czytnika kart MicroSD.
- **Interfejs UART2 (TX=P17, RX=P16):** Dedykowane połączenie komunikacyjne z czytnikiem linii papilarnych R307 (z wykorzystaniem dzielnika napięcia na linii RX mikrokontrolera).
- **GPIO Sterujące:** Bezpośrednie połączenia do diod LED (P13, P12, P4), buzzera (P14) oraz enkodera (P25, P26, P27).

Opis montażu układu

Płytkę PCB należy zamontować z wykorzystaniem śrub M3 wraz z podkładkami w odpowiednich otworach montażowych znajdujących się miejscu obudowy. Następnie należy podłączyć odpowiednio moduły do mikrokontrolera i do odpowiadających im linii zasilania. Na płytce na jednym końcu znajdują się 2 linie zasilania. Odpowiednio 3.3V oraz 5V – o grubości ścieżki dostosowanej do wymagań układu wg. Analizy elementów. Po przeciwnej stronie płytki znajduje się pole masy GND.

Moduły enkodera, wyświetlacza OLED, adaptera MicroSD oraz diod w rzeczywistym układzie nie są bezpośrednio przymocowane do płytki PCB - tylko do obudowy układu z wykorzystaniem izolowanej taśmy dwustronnej piankowej (dedykowanej do tego typu układów).

Na samej płytce PCB wszystkie elementy modułowe, w tym ESP32, RTC, Buzzer – zostały osadzone w socketach (headerach) żeńskich co pozwala na łatwą wymianę modułów w razie awarii. Podobnie moduły nieosadzone na PCB - one również mają swoje piny połączone (z płytką PCB) kablami zakończonymi odpowiednimi headerami – co pozwala na łatwą wymianę modułów.

Bateria systemu RTC DS1307 została umieszczona częścią baterijną do zewnątrz – co ułatwia potencjalną wymianę baterii.

Wszystkie wyprowadzenia zewnętrzne, otwory, sloty – zostały zabezpieczone przeciwko działaniom zewnętrznego środowiska – w tym kurzu, odpowiednią warstwą pianki izolacyjnej. Zapewnia to również dodatkowy aspekt estetyczny – maskując wszystkie nierówności powstałe w wyniku ręcznej obróbki otworów w obudowie uniwersalnej.

Opis programowania układu

Układ jest programowany za pomocą narzędzi wbudowanych w środowisko ESP IDF, za pośrednictwem portu USB znajdującego się na module mikrokontrolera. Przez cały cykl rozwoju oprogramowania również wykorzystywano środowisko ESP IDF.

Opis sposobu uruchamiania oraz testowania

A. Uruchomienie układu

Procedura pierwszego uruchomienia urządzenia wymaga przygotowania nośnika danych oraz zapewnienia zasilania dla podtrzymania czasu.

1. Przygotowanie pamięci masowej:

- Kartę MicroSD należy sformatować w systemie plików FAT32.
- W katalogu głównym karty należy utworzyć plik config.txt zawierający podstawową konfigurację (np. `work=60`, `break=15`).
- (Opcjonalnie) Aby ustawić aktualną datę w module RTC, w pliku konfiguracyjnym należy dodać linię `set_time=1` oraz `datetime=RRRR-MM-DD GG:MM:SS`.

2. Montaż baterii RTC: W gnieździe modułu DS1307 należy umieścić baterię litową CR2032 (3V), zwracając uwagę na polaryzację. Zapewni to podtrzymanie zegara po odłączeniu zasilania głównego.

3. Instalacja karty: Przygotowaną kartę MicroSD należy umieścić w slotcie czytnika urządzenia (do momentu kliknięcia mechanizmu zatrzaskowego).

4. Podłączenie zasilania: Urządzenie należy podłączyć do źródła zasilania (port USB komputera lub ładowarka 5V) za pomocą przewodu microUSB.

5. Weryfikacja inicjalizacji:

- Bezpośrednio po podłączeniu zasilania, dioda zielona powinna mignąć, a buzzer wydać krótki dźwięk (sygnał **signal_ok** w kodzie).
- Na wyświetlaczu OLED powinien pojawić się ekran startowy z napisem "SYSTEM RCP - Gotowy" oraz aktualną godziną pobraną z RTC.
- Jeśli na ekranie pojawi się komunikat błędu (np. dotyczący karty SD lub czujnika), należy odłączyć zasilanie i sprawdzić połączenia.

B. Testowanie funkcjonalności

Po poprawnym uruchomieniu przeprowadzono serię testów weryfikujących zgodność działania z założeniami projektowymi.

a. Test rejestracji i weryfikacji biometrycznej

- **Cel:** Sprawdzenie poprawności komunikacji z czujnikiem R307 i zapisu szablonów.
- **Przebieg:** Wywołano tryb administratora (poprzez obrót enkodera w stanie spoczynku). Wybrano opcję "Dodaj Usera". Zgodnie z instrukcjami na ekranie przyłożono palec dwukrotnie.
- **Wynik:** System wyświetlił komunikat "Zapisano!" i nadał użytkownikowi ID. Przy ponownym przyłożeniu tego samego palca w trybie logowania, system poprawnie rozpoznał tożsamość, zapalając zieloną diodę. Próba przyłożenia niezarejestrowanego palca skutkowała czerwoną diodą i komunikatem błędu.

b. Test logowania czasu pracy i zapisu na kartę SD

- **Cel:** Weryfikacja poprawności zapisu logów w pliku **data.csv**.
- **Przebieg:** Zalogowano się jako zarejestrowany użytkownik i wybrano tryb "PRACA". Po upływie minuty wylogowano się ręcznie.
- **Wynik:** Po wyjęciu karty SD i odczytaniu jej na komputerze, plik **data.csv** zawierał poprawne wpisy z datą, godziną, ID użytkownika oraz typem zdarzenia (START_PRACA, WYLOGOWANIE_RECZNE).

c. Test alarmów i limitów czasu

- **Cel:** Sprawdzenie działania timera tła i sygnalizacji alarmowej.
- **Przebieg:** Na karcie SD zmodyfikowano plik **config.txt**, ustawiając bardzo krótki limit czasu pracy (**work=1 minuta**). Uruchomiono tryb pracy.
- **Wynik:** Po upływie jednej minuty urządzenie zablokowało interfejs, uruchomiło sygnalizację świetlną (miganie diod) oraz dźwiękową (buzzer). Alarm został wyłączony dopiero po ponownym przyłożeniu palca przez użytkownika, co automatycznie zmieniło jego status na "PRZERWA".

d. Test podtrzymania zegara (RTC)

- **Cel:** Weryfikacja działania baterii CR2032 i modułu DS1307.
- **Przebieg:** Odłączono zasilanie USB od urządzenia na okres 1 godziny.
- **Wynik:** Po ponownym podłączeniu zasilania, system wyświetlił poprawną, aktualną godzinę (z uwzględnieniem upływu czasu braku zasilania), co potwierdza poprawność działania obwodu RTC.

Wnioski i uwagi z przebiegu pracy

Podczas realizacji projektu napotkano szereg wyzwań natury projektowej oraz programistycznej, które wymagały analizy i wdrożenia działań naprawczych.

Dopasowanie poziomów logicznych: Ze względu na fakt, że czytnik linii papilarnych R307 pracuje w logice 5V, a mikrokontroler ESP32 w logice 3.3V, konieczne było zabezpieczenie pinu wejściowego (RX) mikrokontrolera. Zastosowano dzielnik napięcia oparty na rezystorach, co skutecznie chroni porty GPIO przed uszkodzeniem, zachowując pewność transmisji UART. Niestety, zakupiony konwerter stanów logicznych nie działał w sposób prawidłowy. Pierwotne rozwiązanie z konwerterem stanów logicznych zostało zastąpione prostszym, tańszym i równie efektywnym rozwiązaniem.

Zarządzanie bibliotekami w środowisku ESP-IDF: Wykorzystanie zaawansowanej biblioteki graficznej **u8g2** znacząco podniosło estetykę interfejsu, ale wygenerowało problem z rozmiarem projektu (folder components przekraczał 500 MB). Było to spowodowane dołączaniem pełnej historii zmian (.git) oraz zbędnych przykładów i czcionek. Przeprowadzono optymalizację struktury plików, usuwając zbędne katalogi, co zredukowało rozmiar źródeł do akceptowalnego poziomu kilkudziesięciu megabajtów. Sama biblioteka również sprawiła ogromny problem przy konfiguracji w środowisku, nie wszystkie fork'i biblioteki na GitHubie obsługują pracę z ESP32, co wymagało dogłębnej analizy przy ściąganiu biblioteki i pochłonęło wiele cennych godzin.

Dopasowanie rozmiarów obudowy do płytki: Przy pierwszych przymiarkach do budowy już fizycznego układu projektu, mimo zgodności wymiarów „na papierze” – dostarczona obudowa była za wąska o ok 2mm, co sprawiło że płytka PCB nie mieściła się w środku. Pomogło podgrzanie obudowy.

Po polutowaniu: Czytnik linii papilarnych nie działał w sposób prawidłowy. Po sprawdzeniu zasilania na linii 5V, masy a także linii TX oraz RX na mierniku uniwersalnym pojawiały się oczekiwane wartości. Rozwiązaniem okazały się błędnie zdefiniowane PIN-y TX oraz RX układu w nowo wgranym kodzie (piny zostały software'owo zamienione miejscami na GPIO ESP32)

W Altiumie: Największym wyzwaniem w środowisku Altium Designer okazało się poprawne zdefiniowanie sieci (Nets). W początkowej fazie projektu wystąpił krytyczny błąd, w którym wylewka masy (Polygon Pour) została błędnie przypisana do sieci pinu buzzera zamiast do GND. Skutkowało to zniknięciem połączeń (ratsnest) i licznymi błędami DRC (Design Rule Check). Problem rozwiązano poprzez ręczną edycję nazw sieci (Net Name) na schemacie oraz wymuszenie pełnej synchronizacji ze strukturą PCB – dodatkowo podczas trasowania ścieżek wystąpił problem "samotnych wysp" (Single Node Nets) przy diodach LED, gdzie program nie wyświetlał linii połączeń mimo poprawnych etykiet. Rozwiązaniem okazało się odświeżenie listy połączeń oraz ręczna weryfikacja przypisań w panelu właściwości padów.

Wnioski końcowe

Zrealizowany projekt "**Biometrycznego systemu rejestracji czasu pracy**" spełnił wszystkie założenia projektowe. Udało się stworzyć w pełni funkcjonalne urządzenie, które integruje w sobie obsługę zaawansowanych peryferiów: biometrii, pamięci masowej, zegara czasu rzeczywistego oraz interfejsu graficznego.

Osiągnięcia:

Poprawność sprzętowa

Zaprojektowana płytką PCB po wyeliminowaniu błędów wieku dziecięcego (problemy z masą) okazała się poprawna elektrycznie. Proces weryfikacji DRC zakończył się wynikiem "0 Errors", a zmontowany układ działa stabilnie.

Stabilność oprogramowania

Implementacja maszyny stanów w pętli głównej pozwala na płynną obsługę interfejsu użytkownika przy jednoczesnym monitorowaniu czasu w tle. System poprawnie reaguje na priorytety (w tym alarmy) i nie zawiesza się podczas zapisu danych na kartę SD.

Edukacja i Satysfakcja Realizacja projektu pozwoliła na praktyczne opanowanie profesjonalnego środowiska Altium Designer (tworzenie schematów, bibliotek, trasowanie PCB, generowanie plików produkcyjnych). Szczególną satysfakcję sprawiło *pierwsze poprawne uruchomienie urządzenia*, gdy po rozwiązaniu problemów z komunikacją, na ekranie OLED pojawił się status "Gotowy", a czytnik poprawnie zweryfikował odcisk palca.

Rozwojowość: Wybór mikrokontrolera ESP32 daje ogromny potencjał na przyszłość. Obecny projekt można łatwo rozbudować o funkcje sieciowe (np. synchronizacja czasu przez NTP, wysyłanie logów do chmury przez Wi-Fi), bez konieczności ingerencji w warstwę sprzętową. Dodatkowo projekt ma potencjał na podłączenie hardware'u do jakiegoś większego projektu z chmurą i bazą danych. Dodatkowo istnieje możliwość stworzenia „sieci” tego typu urządzeń w jakiejś organizacji (np. firmie, szkole) i zarządzania niej z poziomu software'u z jednego miejsca. Obserwując statusy użytkowników i ich postępy w konkretnych projektach, celach – z jednego miejsca.

Projekt pokazuje, że przy odpowiedniej analizie i determinacji w rozwiązywaniu problemów, możliwe jest stworzenie dobrze wyglądającego i działającego systemu wbudowanego – niedużym kosztem.

Literatura:

ESP32 Series Datasheet Version 5.2

https://documentation.espressif.com/esp32_datasheet_en.pdf

Espressif Systems, ESP-IDF Programming Guide, dostęp online:

<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>

Maxim Integrated, DS1307 64 x 8, Serial, I2C Real-Time Clock Datasheet, Rev 10

<https://www.analog.com/media/en/technical-documentation/data-sheets/ds1307.pdf>

Hangzhou Grow Technology Co., Ltd, R307 Optical Fingerprint Module User Manual.

<https://robu.in/wp-content/uploads/2024/08/R307S-fingerprint-module-user-manual.pdf>

Altium Limited, *Altium Designer Documentation*, dostęp online:
<https://www.altium.com/documentation>

Sino Wealth, *SH1106 132 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller*
https://cdn.velleman.eu/downloads/29/infosheets/sh1106_datasheet.pdf